

```
GOOGLE_MAPS_API_KEY = "AIzaSyC0Eo65FvdvFDC0chQq9AWDDyJehssMxdY"
```

```
print("API key loaded")
```

```
API key loaded
```

```
# =====
# CELL 2 (REPLACE YOUR EXISTING CELL 2) -
# GOOGLE SATELLITE MAP + 4-CORNER BOUNDARY + OBSTACLE MARKING
# =====
from google.colab import output
from IPython.display import display, HTML
import json

LAT = 22.7196 # Indore
LON = 75.8577
ZOOM = 19

html = f"""
<!DOCTYPE html>
<html>
<head>
<style>
html, body {{ margin: 0; padding: 0; width: 100%; height: 100%; }}
#map {{ width: 100%; height: 650px; }}
#panel {{
  position: absolute; top: 15px; left: 15px; z-index: 999;
  background: white; padding: 15px; border-radius: 10px;
  font-family: Arial; box-shadow: 0 2px 10px rgba(0,0,0,0.3);
  max-width: 260px;
}}
button {{ margin-top: 8px; padding: 9px 13px; border: none; border-radius: 6px; cursor: pointer; font-size: 13px; }}
#submit {{ background: #16a34a; color: white; }}
#reset {{ background: #dc2626; color: white; }}
#addObstacle {{ background: #ea580c; color: white; }}
#finishObstacle {{ background: #ea580c; color: white; opacity: 0.6; }}
#undoObstacle {{ background: #6b7280; color: white; }}
.hint {{ font-size: 12px; color: #555; margin-top: 6px; }}
</style>
</head>
<body>
<div id="panel">
  <b>Parking Space Optimizer</b>
  <br><br>
  <b>Step 1</b> - click the <b>4 corners</b> of the parking plot.
  <br>Boundary: <span id="bcount">0</span> / 4
  <br><br>
  <b>Step 2 (optional)</b> - mark obstacles (trees, poles, structures).
  <br>Obstacles marked: <span id="ocount">0</span>
  <br>
  <button id="addObstacle" onclick="startObstacle()">+ ADD OBSTACLE</button>
  <button id="finishObstacle" onclick="finishObstacle()">FINISH OBSTACLE</button>
  <button id="undoObstacle" onclick="undoObstacle()">UNDO LAST OBSTACLE</button>
  <div class="hint" id="modeHint">Finish the boundary first.</div>
  <br>
  <button id="submit" onclick="submitAll()">SUBMIT AREA</button>
  <button id="reset" onclick="resetAll()">RESET</button>
</div>
<div id="map"></div>
<script>
let map;
let boundaryPoints = [];
let boundaryMarkers = [];
let boundaryPolygon = null;

let obstacles = []; // finished obstacles: array of point arrays
let obstacleOverlays = []; // finished obstacle polygon overlays

let currentObstaclePoints = [];
let currentObstacleMarkers = [];
let currentObstaclePolygon = null;
let drawingObstacle = false;

function initMap() {{
  const center = {{ lat: {LAT}, lng: {LON} }};
```

```

map = new google.maps.Map(document.getElementById("map"), {{
  center: center, zoom: {ZOOM}, mapTypeId: "satellite",
  tilt: 0, streetViewControl: false, fullscreenControl: true, mapTypeControl: true
}});

map.addListener("click", function(event) {{
  const point = {{ lat: event.latLng.lat(), lng: event.latLng.lng() }};

  if (drawingObstacle) {{
    currentObstaclePoints.push(point);
    const m = new google.maps.Marker({{
      position: point, map: map,
      icon: {{ path: google.maps.SymbolPath.CIRCLE, scale: 5, fillColor: "#ea580c",
        fillOpacity: 1, strokeColor: "#fff", strokeWeight: 1 }}
    }});
    currentObstacleMarkers.push(m);
    if (currentObstaclePolygon) currentObstaclePolygon.setMap(null);
    if (currentObstaclePoints.length >= 3) {{
      currentObstaclePolygon = new google.maps.Polygon({{
        paths: currentObstaclePoints, strokeColor: "#ea580c", strokeOpacity: 1,
        strokeWeight: 2, fillColor: "#ea580c", fillOpacity: 0.30, map: map
      }});
    }}
    return;
  }}

  if (boundaryPoints.length >= 4) return;
  boundaryPoints.push(point);
  const marker = new google.maps.Marker({{ position: point, map: map, label: String(boundaryPoints.length) }});
  boundaryMarkers.push(marker);
  document.getElementById("bcount").innerText = boundaryPoints.length;

  if (boundaryPoints.length === 4) {{
    boundaryPolygon = new google.maps.Polygon({{
      paths: boundaryPoints, strokeColor: "#00ff00", strokeOpacity: 1,
      strokeWeight: 3, fillColor: "#00ff00", fillOpacity: 0.20, map: map
    }});
    document.getElementById("modeHint").innerText = "Boundary done. You can add obstacles now.";
  }}
}});
}}

function startObstacle() {{
  if (boundaryPoints.length < 4) {{
    alert("Finish the 4-corner boundary first.");
    return;
  }}
  drawingObstacle = true;
  currentObstaclePoints = [];
  currentObstacleMarkers = [];
  document.getElementById("modeHint").innerText = "Click 3+ points around the obstacle, then FINISH OBSTACLE.";
}}

function finishObstacle() {{
  if (!drawingObstacle) return;
  if (currentObstaclePoints.length < 3) {{
    alert("Click at least 3 points to define an obstacle.");
    return;
  }}
  obstacles.push(currentObstaclePoints);
  if (currentObstaclePolygon) obstacleOverlays.push(currentObstaclePolygon);
  currentObstaclePoints = [];
  currentObstacleMarkers = [];
  currentObstaclePolygon = null;
  drawingObstacle = false;
  document.getElementById("ocount").innerText = obstacles.length;
  document.getElementById("modeHint").innerText = "Obstacle added. Add another or submit.";
}}

function undoObstacle() {{
  if (obstacles.length === 0) return;
  obstacles.pop();
  const overlay = obstacleOverlays.pop();
  if (overlay) overlay.setMap(null);
  document.getElementById("ocount").innerText = obstacles.length;
}}

```

```

function resetAll() {{
  boundaryPoints = [];
  boundaryMarkers.forEach(m => m.setMap(null));
  boundaryMarkers = [];
  if (boundaryPolygon) {{ boundaryPolygon.setMap(null); boundaryPolygon = null; }}
  obstacles = [];
  obstacleOverlays.forEach(o => o.setMap(null));
  obstacleOverlays = [];
  currentObstaclePoints = [];
  currentObstacleMarkers.forEach(m => m.setMap(null));
  currentObstacleMarkers = [];
  if (currentObstaclePolygon) {{ currentObstaclePolygon.setMap(null); currentObstaclePolygon = null; }}
  drawingObstacle = false;
  document.getElementById("bcount").innerText = "0";
  document.getElementById("ocount").innerText = "0";
  document.getElementById("modeHint").innerText = "Finish the boundary first.";
}}

function submitAll() {{
  if (boundaryPoints.length !== 4) {{
    alert("Please select exactly 4 boundary corners.");
    return;
  }}
  google.colab.kernel.invokeFunction(
    "parking.receive_data",
    [{ boundary: boundaryPoints, obstacles: obstacles }],
    {}
  );
}}
</script>
<script async src="https://maps.googleapis.com/maps/api/js?key={GOOGLE_MAPS_API_KEY}&loading=async&callback=initMap"></script>
</body>
</html>
"""

received_data = None

def receive_data(data):
  global received_data
  received_data = data
  print("=====")
  print("PARKING AREA + OBSTACLES SELECTED")
  print("=====")
  for i, p in enumerate(data["boundary"]):
    print(f"P{i+1}: lat={p['lat']:.8f}, lng={p['lng']:.8f}")
  print(f"\nObstacles marked: {len(data['obstacles'])}")
  for i, obs in enumerate(data["obstacles"]):
    print(f"  Obstacle {i+1}: {len(obs)} points")

output.register_callback("parking.receive_data", receive_data)
display(HTML(html))

```

Parking Space Optimizer

Step 1 — click the 4 corners of the parking plot.
Boundary: 4 / 4

Step 2 (optional) — mark obstacles (trees, poles, structures).
Obstacles marked: 0

+ ADD OBSTACLE

FINISH OBSTACLE

UNDO LAST OBSTACLE

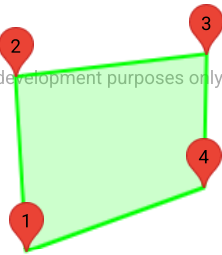
Boundary done. You can add obstacles now.

SUBMIT AREA RESET

For development purposes only

For development purposes only

For development purpo:



For development purposes only

For development purposes only

For development purpo:



For development purposes only

For development purposes only

For development purposes only

For development purpo:

Imagery ©2026 Airbus, CNES / Airbus, Maxar Technologies Report a map error

```
=====
PARKING AREA + OBSTACLES SELECTED
=====
P1: lat=22.71981772, lng=75.85805852
P2: lat=22.72014182, lng=75.85803706
P3: lat=22.72018388, lng=75.85842062
P4: lat=22.71993648, lng=75.85841526
```

```
Obstacles marked: 0
=====
PARKING AREA + OBSTACLES SELECTED
=====
P1: lat=22.71981772, lng=75.85805852
P2: lat=22.72014182, lng=75.85803706
P3: lat=22.72018388, lng=75.85842062
P4: lat=22.71993648, lng=75.85841526
```

```
Obstacles marked: 0
=====
PARKING AREA + OBSTACLES SELECTED
=====
P1: lat=22.71981772, lng=75.85805852
P2: lat=22.72014182, lng=75.85803706
P3: lat=22.72018388, lng=75.85842062
P4: lat=22.71993648, lng=75.85841526
```

Obstacles marked: 0

```
# =====
# CELL 3 (REPLACE YOUR EXISTING CELL 3) -
# GEO -> METRIC GEOMETRY, NOW OBSTACLE-AWARE
# =====
!pip -q install pyproj shapely

import math
import numpy as np
from pyproj import Transformer
from shapely.geometry import Polygon
from shapely.ops import unary_union

# =====
# CHECK INPUT
# =====
```

```

if received_data is None:
    raise RuntimeError("Run Cell 2 and submit the boundary first.")

boundary_points = received_data["boundary"]
obstacle_point_lists = received_data["obstacles"]

# =====
# WGS84 BOUNDARY
# =====
lats = np.array([p["lat"] for p in boundary_points])
lons = np.array([p["lng"] for p in boundary_points])

# =====
# AUTOMATIC UTM ZONE
# =====
mean_lon = float(np.mean(lons))
mean_lat = float(np.mean(lats))
utm_zone = int(math.floor((mean_lon + 180) / 6) + 1)
epsg = (32600 if mean_lat >= 0 else 32700) + utm_zone

print("=====")
print("COORDINATE SYSTEM")
print("=====")
print(f"Mean latitude : {mean_lat:.8f}")
print(f"Mean longitude : {mean_lon:.8f}")
print(f"UTM zone : {utm_zone}")
print(f"EPSG : {epsg}")

transformer = Transformer.from_crs("EPSG:4326", f"EPSG:{epsg}", always_xy=True)

def to_utm(points):
    return [transformer.transform(p["lng"], p["lat"]) for p in points]

# =====
# BOUNDARY POLYGON
# =====
utm_points = to_utm(boundary_points)
PARKING_POLYGON_UTM = Polygon(utm_points)
if not PARKING_POLYGON_UTM.is_valid:
    PARKING_POLYGON_UTM = PARKING_POLYGON_UTM.buffer(0)

PARKING_AREA_M2 = PARKING_POLYGON_UTM.area
PARKING_PERIMETER_M = PARKING_POLYGON_UTM.length

# =====
# OBSTACLE POLYGONS (trees, poles, structures, etc.)
# =====
OBSTACLE_POLYGONS_UTM = []
for obs_points in obstacle_point_lists:
    if len(obs_points) < 3:
        continue
    obs_utm = to_utm(obs_points)
    obs_poly = Polygon(obs_utm)
    if not obs_poly.is_valid:
        obs_poly = obs_poly.buffer(0)
    # keep only the part that is actually inside the plot
    obs_poly = obs_poly.intersection(PARKING_POLYGON_UTM)
    if not obs_poly.is_empty:
        OBSTACLE_POLYGONS_UTM.append(obs_poly)

OBSTACLES_UNION_UTM = (
    unary_union(OBSTACLE_POLYGONS_UTM) if OBSTACLE_POLYGONS_UTM else None
)

# =====
# USABLE POLYGON = BOUNDARY MINUS OBSTACLES
# This is what the optimizer (Cell 4) should actually search over.
# PARKING_POLYGON_UTM stays as the true site boundary for display/area.
# =====
if OBSTACLES_UNION_UTM is not None:
    PARKING_POLYGON_UTM_USABLE = PARKING_POLYGON_UTM.difference(OBSTACLES_UNION_UTM)
else:
    PARKING_POLYGON_UTM_USABLE = PARKING_POLYGON_UTM

OBSTACLE_AREA_M2 = OBSTACLES_UNION_UTM.area if OBSTACLES_UNION_UTM is not None else 0.0
USABLE_AREA_M2 = PARKING_POLYGON_UTM_USABLE.area

print("\n=====")

```

```

print("PARKING GEOMETRY")
print("=====")
print(f"Plot area      : {PARKING_AREA_M2:.2f} m²")
print(f"Perimeter      : {PARKING_PERIMETER_M:.2f} m")
print(f"Obstacles marked : {len(OBSTACLE_POLYGONS_UTM)}")
print(f"Obstacle area   : {OBSTACLE_AREA_M2:.2f} m²")
print(f"Usable area     : {USABLE_AREA_M2:.2f} m²")

if PARKING_POLYGON_UTM_USABLE.geom_type != "Polygon":
    print(
        "\n△ Obstacles split the usable area into multiple pieces "
        f"({PARKING_POLYGON_UTM_USABLE.geom_type}). "
        "Cell 4 will optimize on the largest piece only – keep this in mind "
        "if a big obstacle cuts the plot in half."
    )
    # Reduce to the single largest polygon so downstream code (which
    # assumes a single Polygon) keeps working.
    if PARKING_POLYGON_UTM_USABLE.geom_type == "MultiPolygon":
        PARKING_POLYGON_UTM_USABLE = max(
            PARKING_POLYGON_UTM_USABLE.geoms, key=lambda g: g.area
        )

# =====
# SAVE
# =====
PARKING_EPSG = epsg
print("\n✓ Accurate, obstacle-aware metric geometry ready.")

```

```

=====
COORDINATE SYSTEM
=====
Mean latitude  : 22.72001998
Mean longitude : 75.85823287
UTM zone      : 43
EPSG          : 32643

=====
PARKING GEOMETRY
=====
Plot area      : 1209.94 m²
Perimeter      : 141.93 m
Obstacles marked : 0
Obstacle area  : 0.00 m²
Usable area    : 1209.94 m²

```

✓ Accurate, obstacle-aware metric geometry ready.

```

# =====
# CELL 4 – RESEARCH-BACKED PARKING OPTIMIZER
# =====

import numpy as np
import math
import matplotlib.pyplot as plt

from shapely.geometry import box
from shapely.affinity import rotate, translate

# =====
# PARKING PROFILE
# =====

# BIS SP 73:2023 shared car parking:
# 2.75 m x 5.00 m

SLOT_WIDTH = 2.75
SLOT_LENGTH = 5.00

# =====
# AISLE PROFILE
# =====

# Engineering defaults for the prototype.
#
# 45° / 60°:
# one-way circulation

```

```

#
# 90°:
# two-way circulation
#
# These are configurable engineering assumptions,
# NOT a substitute for local authority approval.

AISLE_WIDTHS = {

    0: 3.60,

    45: 4.80,

    60: 4.80,

    90: 6.00
}

def get_aisle_width(angle):

    """
    Interpolate between standard anchor values.
    """

    known_angles = np.array([
        0,
        45,
        60,
        90
    ])

    known_widths = np.array([
        3.60,
        4.80,
        4.80,
        6.00
    ])

    return float(
        np.interp(
            angle,
            known_angles,
            known_widths
        )
    )

# =====
# ANGLE SEARCH
# =====

ANGLE_STEP = 1.0

ANGLES = np.arange(
    0,
    90.01,
    ANGLE_STEP
)

# =====
# GEOMETRY HELPERS
# =====

def slot_geometry(
    x,
    y,
    angle
):

    """
    Creates a parking slot centered at x,y.

    Parking angle:
    90° = perpendicular
    45° = diagonal

```

```

        0° = parallel
        """

    slot = box(
        -SLOT_WIDTH / 2,
        -SLOT_LENGTH / 2,

        SLOT_WIDTH / 2,
        SLOT_LENGTH / 2
    )

    # Convert parking angle to rectangle rotation.
    rotation = 90 - angle

    slot = rotate(
        slot,
        rotation,
        origin="center",
        use_radians=False
    )

    slot = translate(
        slot,
        xoff=x,
        yoff=y
    )

    return slot

# =====
# PLOT ORIENTATION
# =====

def longest_edge_orientation(
    polygon
):
    coords = list(
        polygon.exterior.coords
    )

    best_length = -1
    best_angle = 0

    for i in range(
        len(coords) - 1
    ):

        x1, y1 = coords[i]

        x2, y2 = coords[i + 1]

        dx = x2 - x1
        dy = y2 - y1

        length = math.hypot(
            dx,
            dy
        )

        angle = math.degrees(
            math.atan2(
                dy,
                dx
            )
        )

        if length > best_length:

            best_length = length
            best_angle = angle

    return best_angle

# =====

```

```

# ROTATE PLOT INTO LOCAL COORDINATES
# =====

DOMINANT_ANGLE = (
    longest_edge_orientation(
        PARKING_POLYGON_UTM
    )
)

print("=====")
print("SITE ORIENTATION")
print("=====")

print(
    f"Dominant edge angle = "
    f"{DOMINANT_ANGLE:.2f}°"
)

LOCAL_POLYGON = rotate(
    PARKING_POLYGON_UTM,
    -DOMINANT_ANGLE,
    origin="centroid",
    use_radians=False
)

# =====
# MODULE DIMENSIONS
# =====

def module_dimensions(
    angle
):
    theta = math.radians(
        angle
    )

    # Perpendicular depth of one stall
    stall_depth = (
        SLOT_LENGTH * math.sin(theta)
        +
        SLOT_WIDTH * math.cos(theta)
    )

    # Length consumed along aisle
    stall_frontage = (
        SLOT_LENGTH * math.cos(theta)
        +
        SLOT_WIDTH * math.sin(theta)
    )

    aisle = get_aisle_width(
        angle
    )

    # Double-sided parking module
    module_depth = (
        stall_depth
        +
        aisle
        +
        stall_depth
    )

    return (
        stall_depth,
        stall_frontage,
        aisle,
        module_depth
    )

# =====
# BUILD ONE BAY

```

```

# =====

def build_bay(
    polygon,
    base_y,
    angle
):
    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    minx, miny, maxx, maxy = (
        polygon.bounds
    )

    # -----
    # AISLE
    # -----

    aisle = box(
        minx,
        base_y + stall_depth,
        maxx,
        base_y + stall_depth + aisle_width
    )

    # Require reasonable aisle coverage
    aisle_intersection = (
        aisle.intersection(
            polygon
        )
    )

    aisle_ratio = (
        aisle_intersection.area
        /
        max(
            aisle.area,
            1e-9
        )
    )

    # -----
    # PARKING ROWS
    # -----

    row1_y = (
        base_y
        +
        stall_depth / 2
    )

    row2_y = (
        base_y
        +
        stall_depth
        +
        aisle_width
        +
        stall_depth / 2
    )

    row1 = []
    row2 = []

    # -----

```

```

# X POSITIONS
# -----

# Slight phase shifts can improve irregular-edge fit.
x_start = (
    minx
    +
    frontage / 2
)

x_positions = np.arange(
    x_start,
    maxx,
    frontage
)

# -----
# FIRST ROW
# -----

for x in x_positions:

    slot = slot_geometry(
        x,
        row1_y,
        angle
    )

    if polygon.covers(
        slot
    ):

        row1.append(
            slot
        )

# -----
# SECOND ROW
# -----

for x in x_positions:

    slot = slot_geometry(
        x,
        row2_y,
        angle
    )

    if polygon.covers(
        slot
    ):

        row2.append(
            slot
        )

return (
    row1,
    row2,
    aisle,
    aisle_ratio,
    module_depth
)

# =====
# COMPLETE LAYOUT
# =====

def generate_layout(
    polygon,
    angle,
    y_offset

```

```

):

    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    minx, miny, maxx, maxy = (
        polygon.bounds

    )

    all_slots = []
    all_aisles = []

    # -----
    # Repeat double-sided modules
    # -----

    y = (
        miny
        +
        y_offset
    )

    while (
        y + module_depth
        <= maxy + 1e-9
    ):

        (
            row1,
            row2,
            aisle,
            aisle_ratio,
            -
        ) = build_bay(
            polygon,
            y,
            angle
        )

        # -----
        # Accept bay only when aisle is mostly usable
        # -----

        if (
            aisle_ratio >= 0.80
            and
            len(row1) > 0
            and
            len(row2) > 0
        ):

            all_slots.extend(
                row1
            )

            all_slots.extend(
                row2
            )

            all_aisles.append(
                aisle
            )

        y += module_depth

```

```

    return (
        all_slots,
        all_aisles
    )

# =====
# SEARCH BEST OFFSET FOR EACH ANGLE
# =====

def optimize_angle(
    polygon,
    angle
):
    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    best_slots = []
    best_aisles = []
    best_offset = 0

    # Search all possible phase offsets.
    #
    # This avoids depending on an arbitrary
    # first-row position.

    offsets = np.linspace(
        0,
        module_depth,
        31
    )

    for offset in offsets:

        slots, aisles = (
            generate_layout(
                polygon,
                angle,
                offset
            )
        )

        if len(slots) > len(
            best_slots
        ):

            best_slots = slots
            best_aisles = aisles
            best_offset = offset

    return {
        "angle": angle,
        "slots": best_slots,
        "aisles": best_aisles,
        "offset": best_offset,
        "aisle_width": aisle_width,
        "module_depth": module_depth
    }

# =====
# RUN OPTIMIZATION
# =====

print("\n=====")

```

```

print("RUNNING ANGLE OPTIMIZATION")
print("=====")

results = []

for angle in ANGLES:

    result = optimize_angle(
        LOCAL_POLYGON,
        float(angle)
    )

    results.append(
        result
    )

    print(
        f"angle:5.1f°"
        f" → "
        f"{len(result['slots']):3d} slots"
    )

# =====
# BEST RESULT
# =====

best = max(
    results,
    key=lambda r: len(
        r["slots"]
    )
)

# =====
# SANITY CHECK
# =====

ECS_AREA = 23.0

area_based_upper_bound = (
    PARKING_AREA_M2
    /
    ECS_AREA
)

print("\n=====")
print("FINAL OPTIMIZATION")
print("=====")

print(
    f"Best angle      : "
    f"{best['angle']:.1f}°"
)

print(
    f"Best slot count  : "
    f"{len(best['slots'])}"
)

print(
    f"Aisle width      : "
    f"{best['aisle_width']:.2f} m"
)

print(
    f"Module depth      : "
    f"{best['module_depth']:.2f} m"
)

print(
    f"Plot area         : "
    f"{PARKING_AREA_M2:.2f} m²"

```

```

    )

    print(
        f"23 m²/ECS check    : "
        f"{area_based_upper_bound:.1f} ECS"
    )

# =====
# TOP 10 CONFIGURATIONS
# =====

print("\n=====")
print("TOP CONFIGURATIONS")
print("=====")

top_results = sorted(
    results,
    key=lambda r: len(
        r["slots"]
    ),
    reverse=True
)

for r in top_results[:10]:

    print(
        f"{r['angle']:5.1f}°"
        f" → "
        f"{len(r['slots']):3d} slots"
    )

# =====
# VISUALIZE
# =====

fig, ax = plt.subplots(
    figsize=(15, 10)
)

# Boundary
bx, by = (
    LOCAL_POLYGON
    .exterior
    .xy
)

ax.plot(
    bx,
    by,
    linewidth=3
)

# Aisles
for aisle in best[
    "aisles"
]:

    ax.fill(
        *aisle.exterior.xy,
        alpha=0.12
    )

    ax.plot(
        *aisle.exterior.xy,
        linestyle="--",
        linewidth=1.5
    )

# Slots
for i, slot in enumerate(

```

```
best["slots"],
start=1
):

    sx, sy = (
        slot.exterior.xy
    )

    ax.fill(
        sx,
        sy,
        alpha=0.35
    )

    ax.plot(
        sx,
        sy,
        linewidth=0.8
    )

    c = slot.centroid

    ax.text(
        c.x,
        c.y,
        str(i),
        fontsize=6,
        ha="center",
        va="center"
    )

ax.set_title(
    "RESEARCH-BACKED PARKING OPTIMIZATION\n"
    f"{len(best['slots'])} slots @ "
    f"{best['angle']:.1f}°",
    fontsize=18,
    fontweight="bold"
)

ax.set_xlabel(
    "X (metres)"
)

ax.set_ylabel(
    "Y (metres)"
)

ax.set_aspect(
    "equal"
)

ax.grid(
    alpha=0.2
)

plt.show()
```



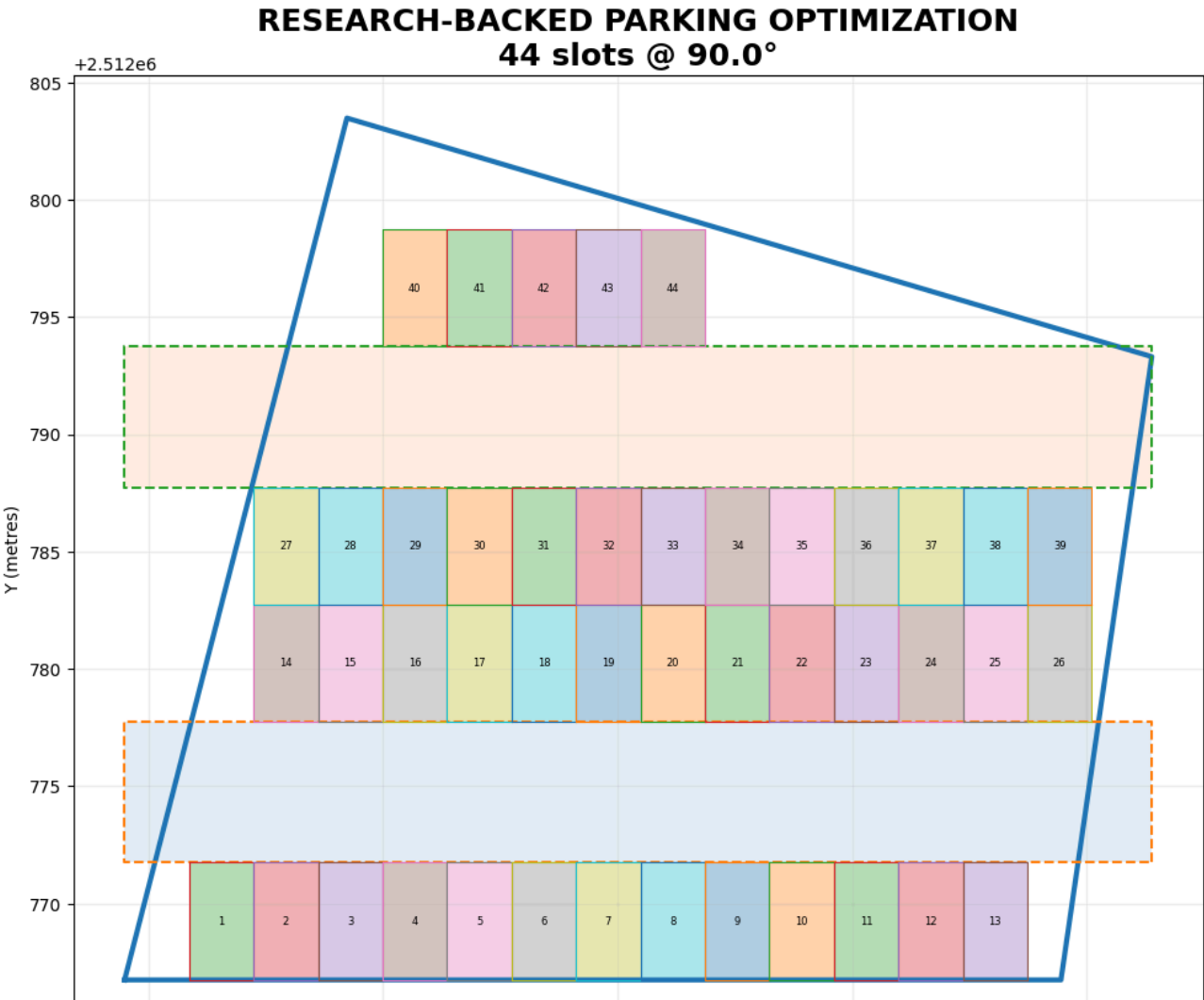
```
=====
SITE ORIENTATION
=====
Dominant edge angle = 9.81°

=====
RUNNING ANGLE OPTIMIZATION
=====
0.0° → 39 slots
1.0° → 38 slots
2.0° → 38 slots
3.0° → 37 slots
4.0° → 36 slots
5.0° → 35 slots
6.0° → 34 slots
7.0° → 34 slots
8.0° → 34 slots
9.0° → 25 slots
10.0° → 24 slots
11.0° → 24 slots
12.0° → 24 slots
13.0° → 24 slots
14.0° → 24 slots
15.0° → 24 slots
16.0° → 24 slots
17.0° → 23 slots
18.0° → 23 slots
19.0° → 23 slots
20.0° → 23 slots
21.0° → 23 slots
22.0° → 23 slots
23.0° → 23 slots
24.0° → 23 slots
25.0° → 23 slots
26.0° → 23 slots
27.0° → 23 slots
28.0° → 23 slots
29.0° → 23 slots
30.0° → 23 slots
31.0° → 23 slots
32.0° → 23 slots
33.0° → 22 slots
34.0° → 22 slots
35.0° → 22 slots
36.0° → 22 slots
37.0° → 22 slots
38.0° → 22 slots
39.0° → 22 slots
40.0° → 22 slots
41.0° → 21 slots
42.0° → 21 slots
43.0° → 21 slots
44.0° → 21 slots
45.0° → 21 slots
46.0° → 21 slots
47.0° → 21 slots
48.0° → 21 slots
49.0° → 21 slots
50.0° → 22 slots
51.0° → 22 slots
52.0° → 22 slots
53.0° → 23 slots
54.0° → 22 slots
55.0° → 22 slots
56.0° → 22 slots
57.0° → 22 slots
58.0° → 23 slots
59.0° → 23 slots
60.0° → 23 slots
61.0° → 23 slots
62.0° → 23 slots
63.0° → 24 slots
64.0° → 24 slots
65.0° → 25 slots
66.0° → 25 slots
67.0° → 25 slots
68.0° → 26 slots
69.0° → 26 slots
70.0° → 26 slots
71.0° → 27 slots
72.0° → 28 slots
73.0° → 27 slots
74.0° → 26 slots
75.0° → 27 slots
-- -- -- -- --
```

```
76.0° → 28 slots
77.0° → 29 slots
78.0° → 30 slots
79.0° → 30 slots
80.0° → 31 slots
81.0° → 32 slots
82.0° → 33 slots
83.0° → 34 slots
84.0° → 36 slots
85.0° → 37 slots
86.0° → 37 slots
87.0° → 40 slots
88.0° → 41 slots
89.0° → 42 slots
90.0° → 44 slots
```

```
=====
FINAL OPTIMIZATION
=====
Best angle      : 90.0°
Best slot count : 44
Aisle width     : 6.00 m
Module depth    : 16.00 m
Plot area       : 1209.30 m²
23 m²/ECS check : 52.6 ECS
```

```
=====
TOP CONFIGURATIONS
=====
90.0° → 44 slots
89.0° → 42 slots
88.0° → 41 slots
87.0° → 40 slots
0.0° → 39 slots
1.0° → 38 slots
2.0° → 38 slots
3.0° → 37 slots
85.0° → 37 slots
86.0° → 37 slots
```



```

# =====
# CELL 4 → CELL 7
# PARKING OPTIMIZER V2
#
# Cell 4 : Parking optimization
# Cell 5 : Entry / Exit + Reachability
# Cell 6 : Google Satellite Overlay
# Cell 7 : Optional Boundary Refinement
# =====

# =====
# =====
# CELL 4 – PARKING OPTIMIZATION
# =====
# =====

import numpy as np
import math
import matplotlib.pyplot as plt

from shapely.geometry import box
from shapely.affinity import rotate, translate

# =====
# PARKING PROFILE
# =====

# BIS/shared parking prototype
SLOT_WIDTH = 2.75
SLOT_LENGTH = 5.00

# =====
# AISLE WIDTHS
# =====

AISLE_WIDTHS = {
    0: 3.60,
    45: 4.80,
    60: 4.80,
    90: 6.00
}

def get_aisle_width(angle):

    known_angles = np.array([
        0,
        45,
        60,
        90
    ])

    known_widths = np.array([
        3.60,
        4.80,
        4.80,
        6.00
    ])

    return float(
        np.interp(
            angle,
            known_angles,
            known_widths
        )
    )

# =====
# ANGLE SEARCH
# =====

ANGLE_STEP = 1.0

```

```

ANGLES = np.arange(
    0,
    90.01,
    ANGLE_STEP
)

# =====
# SLOT GEOMETRY
# =====

def slot_geometry(
    x,
    y,
    angle
):
    slot = box(
        -SLOT_WIDTH / 2,
        -SLOT_LENGTH / 2,
        SLOT_WIDTH / 2,
        SLOT_LENGTH / 2
    )

    rotation = 90 - angle

    slot = rotate(
        slot,
        rotation,
        origin="center",
        use_radians=False
    )

    slot = translate(
        slot,
        xoff=x,
        yoff=y
    )

    return slot

# =====
# FIND DOMINANT SITE ORIENTATION
# =====

def longest_edge_orientation(
    polygon
):
    coords = list(
        polygon.exterior.coords
    )

    best_length = -1
    best_angle = 0

    for i in range(
        len(coords) - 1
    ):
        x1, y1 = coords[i]
        x2, y2 = coords[i + 1]

        dx = x2 - x1
        dy = y2 - y1

        length = math.hypot(
            dx,
            dy
        )

        angle = math.degrees(
            math.atan2(
                dy,
                dx
            )
        )

```

```

    )

    if length > best_length:

        best_length = length
        best_angle = angle

    return best_angle

# =====
# SITE ORIENTATION
# =====

DOMINANT_ANGLE = longest_edge_orientation(
    PARKING_POLYGON_UTM
)

# IMPORTANT:
# Explicit pivot point for reversible rotation
POLY_CENTROID = PARKING_POLYGON_UTM.centroid

print("=====")
print("SITE ORIENTATION")
print("=====")

print(
    f"Dominant edge angle = "
    f"{DOMINANT_ANGLE:.2f}°"
)

# =====
# ROTATE SITE INTO LOCAL COORDINATES
# =====

LOCAL_POLYGON = rotate(
    PARKING_POLYGON_UTM_USABLE,
    -DOMINANT_ANGLE,
    origin=POLY_CENTROID,
    use_radians=False
)

# =====
# MODULE DIMENSIONS
# =====

def module_dimensions(
    angle
):
    theta = math.radians(
        angle
    )

    stall_depth = (
        SLOT_LENGTH * math.sin(theta)
        +
        SLOT_WIDTH * math.cos(theta)
    )

    stall_frontage = (
        SLOT_LENGTH * math.cos(theta)
        +
        SLOT_WIDTH * math.sin(theta)
    )

    aisle = get_aisle_width(
        angle
    )

    module_depth = (
        stall_depth
        +
        aisle
    )

```

```

        stall_depth
    )

    return (
        stall_depth,
        stall_frontage,
        aisle,
        module_depth
    )

# =====
# BUILD ONE DOUBLE-SIDED BAY
# =====

def build_bay(
    polygon,
    base_y,
    angle
):
    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    minx, miny, maxx, maxy = (
        polygon.bounds
    )

    # -----
    # AISLE
    # -----

    aisle = box(
        minx,
        base_y + stall_depth,
        maxx,
        base_y + stall_depth + aisle_width
    )

    aisle_intersection = (
        aisle.intersection(
            polygon
        )
    )

    aisle_ratio = (
        aisle_intersection.area
        /
        max(
            aisle.area,
            1e-9
        )
    )

    # -----
    # PARKING ROW POSITIONS
    # -----

    row1_y = (
        base_y
        +
        stall_depth / 2
    )

    row2_y = (
        base_y
        +

```

```

        stall_depth
        +
        aisle_width
        +
        stall_depth / 2
    )

    row1 = []
    row2 = []

    # -----
    # SLOT X POSITIONS
    # -----

    x_start = (
        minx
        +
        frontage / 2
    )

    x_positions = np.arange(
        x_start,
        maxx,
        frontage
    )

    # -----
    # FIRST ROW
    # -----

    for x in x_positions:

        slot = slot_geometry(
            x,
            row1_y,
            angle
        )

        if polygon.covers(
            slot
        ):

            row1.append(
                slot
            )

    # -----
    # SECOND ROW
    # -----

    for x in x_positions:

        slot = slot_geometry(
            x,
            row2_y,
            angle
        )

        if polygon.covers(
            slot
        ):

            row2.append(
                slot
            )

    return (
        row1,
        row2,
        aisle,
        aisle_ratio,
        module_depth
    )

```

```

# =====
# GENERATE COMPLETE LAYOUT
# =====

def generate_layout(
    polygon,
    angle,
    y_offset
):
    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    minx, miny, maxx, maxy = (
        polygon.bounds
    )

    all_slots = []
    all_aisles = []

    y = (
        miny
        +
        y_offset
    )

    while (
        y + module_depth
        <= maxx + 1e-9
    ):
        (
            row1,
            row2,
            aisle,
            aisle_ratio,
        ) = build_bay(
            polygon,
            y,
            angle
        )

        if (
            aisle_ratio >= 0.80
            and
            len(row1) > 0
            and
            len(row2) > 0
        ):
            all_slots.extend(
                row1
            )

            all_slots.extend(
                row2
            )

            all_aisles.append(
                aisle
            )

        y += module_depth

```

```

    return (
        all_slots,
        all_aisles
    )

# =====
# OPTIMIZE ONE ANGLE
# =====

def optimize_angle(
    polygon,
    angle
):
    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    best_slots = []
    best_aisles = []
    best_offset = 0

    offsets = np.linspace(
        0,
        module_depth,
        31
    )

    for offset in offsets:

        slots, aisles = (
            generate_layout(
                polygon,
                angle,
                offset
            )
        )

        if len(slots) > len(
            best_slots
        ):

            best_slots = slots
            best_aisles = aisles
            best_offset = offset

    return {
        "angle": angle,
        "slots": best_slots,
        "aisles": best_aisles,
        "offset": best_offset,
        "aisle_width": aisle_width,
        "module_depth": module_depth
    }

# =====
# RUN ALL ANGLES
# =====

print("\n=====")
print("RUNNING ANGLE OPTIMIZATION")
print("=====")

```

```

results = []

for angle in ANGLES:

    result = optimize_angle(
        LOCAL_POLYGON,
        float(angle)
    )

    results.append(
        result
    )

    print(
        f"angle:5.1f}°"
        f" → "
        f"{len(result['slots']):3d} slots"
    )

# =====
# BEST RESULT
# =====

best = max(
    results,
    key=lambda r: len(
        r["slots"]
    )
)

# =====
# AREA SANITY CHECK
# =====

ECS_AREA = 23.0

area_based_upper_bound = (
    PARKING_AREA_M2
    /
    ECS_AREA
)

print("\n=====")
print("OPTIMAL RESULT")
print("=====")

print(
    f"Best angle      : "
    f"{best['angle']:.1f}°"
)

print(
    f"Best slot count  : "
    f"{len(best['slots'])}"
)

print(
    f"Aisle width     : "
    f"{best['aisle_width']:.2f} m"
)

print(
    f"Module depth    : "
    f"{best['module_depth']:.2f} m"
)

print(
    f"Plot area       : "
    f"{PARKING_AREA_M2:.2f} m²"
)

print(
    f"23 m²/ECS check : "
    f"{area_based_upper_bound:.1f}"
)

```

```

)

# =====
# TOP CONFIGURATIONS
# =====

print("\n=====")
print("TOP CONFIGURATIONS")
print("=====")

top_results = sorted(
    results,
    key=lambda r: len(
        r["slots"]
    ),
    reverse=True
)

for r in top_results[:10]:

    print(
        f"{r['angle']:5.1f}°"
        f" → "
        f"{len(r['slots']):3d} slots"
    )

# =====
# CELL 4 VISUALIZATION
# =====

fig, ax = plt.subplots(
    figsize=(15, 10)
)

# Boundary
bx, by = (
    LOCAL_POLYGON
    .exterior
    .xy
)

ax.plot(
    bx,
    by,
    linewidth=3
)

# Aisles
for aisle in best[
    "aisles"
]:

    ax.fill(
        *aisle.exterior.xy,
        alpha=0.12
    )

    ax.plot(
        *aisle.exterior.xy,
        linestyle="--",
        linewidth=1.5
    )

# Slots
for i, slot in enumerate(
    best["slots"],
    start=1
):

    sx, sy = (

```

```

        slot.exterior.xy
    )

    ax.fill(
        sx,
        sy,
        alpha=0.35
    )

    ax.plot(
        sx,
        sy,
        linewidth=0.8
    )

    c = slot.centroid

    ax.text(
        c.x,
        c.y,
        str(i),
        fontsize=6,
        ha="center",
        va="center"
    )

ax.set_title(
    "RESEARCH-BACKED PARKING OPTIMIZATION\n"
    f"{len(best['slots'])} slots @ "
    f"{best['angle']:.1f}°",
    fontsize=18,
    fontweight="bold"
)

ax.set_xlabel(
    "X (metres)"
)

ax.set_ylabel(
    "Y (metres)"
)

ax.set_aspect(
    "equal"
)

ax.grid(
    alpha=0.2
)

plt.show()

# =====
# =====
# CELL 5 – ENTRY / EXIT + REACHABILITY
# =====
# =====

from shapely.geometry import Point
from shapely.ops import unary_union

# =====
# ACCESS LANE
# =====

ACCESS_LANE_WIDTH = 6.0

TOUCH_TOLERANCE = 0.05

# =====
# ACCESS-SPINE FUNCTION
# =====

def apply_access_spine(

```

```

    polygon,
    slots,
    aisles,
    side,
    lane_width
):

    minx, miny, maxx, maxy = (
        polygon.bounds
    )

    # -----
    # LEFT ACCESS
    # -----

    if side == "left":

        spine = box(
            minx,
            miny,
            minx + lane_width,
            maxy
        )

    # -----
    # RIGHT ACCESS
    # -----

    else:

        spine = box(
            maxx - lane_width,
            miny,
            maxx,
            maxy
        )

    # -----
    # Slots cannot occupy access spine
    # -----

    kept_slots = [
        s
        for s in slots
        if not s.intersects(
            spine
        )
    ]

    # -----
    # Find aisles connected to spine
    # -----

    connected_aisles = [
        a
        for a in aisles
        if a.intersects(
            spine
        )
    ]

    connected_union = (
        unary_union(
            connected_aisles
        )
        if connected_aisles
        else None
    )

    # -----
    # Find reachable slots

```

```

# -----

reachable_slots = []

if connected_union is not None:

    for s in kept_slots:

        if (
            s.distance(
                connected_union
            )
            <
            TOUCH_TOLERANCE
        ):

            reachable_slots.append(
                s
            )

# -----
# Entry point
# -----

if side == "left":

    entry_point = Point(
        minx,
        (miny + maxy) / 2
    )

else:

    entry_point = Point(
        maxx,
        (miny + maxy) / 2
    )

return {

    "side": side,

    "spine": spine,

    "slots": reachable_slots,

    "aisles": connected_aisles,

    "entry_point": entry_point,

    "n_reachable":
        len(reachable_slots),

    "n_connected_aisles":
        len(connected_aisles),

    "n_total_aisles":
        len(aisles)
}

# =====
# TEST BOTH SIDES
# =====

print("\n=====")
print("ENTRY / EXIT + REACHABILITY")
print("=====")

candidates = []

for side in [
    "left",

```

```

    "right"
]:

    candidate = apply_access_spine(
        LOCAL_POLYGON,
        best["slots"],
        best["aisles"],
        side,
        ACCESS_LANE_WIDTH
    )

    candidates.append(
        candidate
    )

    print(
        f"Access from {side:5s} edge → "
        f"{candidate['n_reachable']:3d} reachable slots "
        f"("
        f"{candidate['n_connected_aisles']}/"
        f"{candidate['n_total_aisles']}"
        f" aisle bays connected)"
    )

# =====
# BEST ACCESS SIDE
# =====

FINAL_CANDIDATE = max(
    candidates,
    key=lambda c:
        c["n_reachable"]
)

# =====
# RESULTS
# =====

print("\n-----")

print(
    f"Best access side      : "
    f"{FINAL_CANDIDATE['side']}"
)

print(
    f"Before reachability   : "
    f"{len(best['slots'])}"
)

print(
    f"After reachability    : "
    f"{FINAL_CANDIDATE['n_reachable']}"
)

dropped = (
    len(best["slots"])
    -
    FINAL_CANDIDATE[
        "n_reachable"
    ]
)

if dropped > 0:

    print(
        f"△ {dropped} slots dropped."
    )

else:

    print(

```

```

        "✓ All slots reachable."
    )

if (
    FINAL_CANDIDATE[
        "n_connected_aisles"
    ]
    <
    FINAL_CANDIDATE[
        "n_total_aisles"
    ]
):

    print(
        "\n△ Some aisle bays are not "
        "connected to the access spine."
    )

# =====
# FINAL LAYOUT
# =====

FINAL_LAYOUT = {

    "angle":
        best["angle"],

    "slots":
        FINAL_CANDIDATE[
            "slots"
        ],

    "aisles":
        FINAL_CANDIDATE[
            "aisles"
        ],

    "spine":
        FINAL_CANDIDATE[
            "spine"
        ],

    "entry_point":
        FINAL_CANDIDATE[
            "entry_point"
        ],

    "access_side":
        FINAL_CANDIDATE[
            "side"
        ]
}

print(
    f"\n✓ FINAL_LAYOUT ready: "
    f"{len(FINAL_LAYOUT['slots'])} "
    f"reachable slots."
)

# =====
# =====
# CELL 6 – SATELLITE OVERLAY
# =====
# =====

from pyproj import Transformer
from shapely.affinity import rotate
from IPython.display import display, HTML
import json

# =====
# UTM → LAT/LON

```

```

# =====
inverse_transformer = (
    Transformer.from_crs(
        f"EPSG:{PARKING_EPSG}",
        "EPSG:4326",
        always_xy=True
    )
)

# =====
# UTM POLYGON → LAT/LON
# =====

def utm_polygon_to_latlon(
    polygon
):
    coords = []

    for x, y in (
        polygon.exterior.coords
    ):
        lon, lat = (
            inverse_transformer.transform(
                x,
                y
            )
        )

        coords.append({
            "lat": lat,
            "lng": lon
        })

    return coords

# =====
# LOCAL → LAT/LON
#
# IMPORTANT:
# Local geometry must first be rotated back
# around EXACT SAME POLY_CENTROID.
# =====

def local_polygon_to_latlon(
    polygon
):
    true_utm = rotate(
        polygon,
        DOMINANT_ANGLE,
        origin=POLY_CENTROID,
        use_radians=False
    )

    return utm_polygon_to_latlon(
        true_utm
    )

# =====
# LOCAL POINT → LAT/LON
# =====

def local_point_to_latlon(
    point
):
    true_utm = rotate(
        point,
        DOMINANT_ANGLE,
        origin=POLY_CENTROID,
        use_radians=False
    )

```

```

        lon, lat = (
            inverse_transformer.transform(
                true_utm.x,
                true_utm.y
            )
        )

        return {
            "lat": lat,
            "lng": lon
        }

# =====
# CONVERT SITE
# =====

site_path = (
    utm_polygon_to_latlon(
        PARKING_POLYGON_UTM
    )
)

# =====
# CONVERT OBSTACLES
# =====

obstacle_paths = [

    utm_polygon_to_latlon(
        obstacle
    )

    for obstacle
    in OBSTACLE_POLYGONS_UTM
]

# =====
# CONVERT FINAL SLOTS
# =====

slot_paths = [

    local_polygon_to_latlon(
        slot
    )

    for slot
    in FINAL_LAYOUT[
        "slots"
    ]
]

# =====
# CONVERT AISLES
# =====

aisle_paths = [

    local_polygon_to_latlon(
        aisle
    )

    for aisle
    in FINAL_LAYOUT[
        "aisles"
    ]
]

# =====
# ACCESS SPINE
# =====

```

```

spine_path = (
    local_polygon_to_latlon(
        FINAL_LAYOUT[
            "spine"
        ]
    )
)

# =====
# ENTRY
# =====

entry_latlon = (
    local_point_to_latlon(
        FINAL_LAYOUT[
            "entry_point"
        ]
    )
)

# =====
# MAP CENTER
# =====

center = (
    PARKING_POLYGON_UTM
    .centroid
)

center_lon, center_lat = (
    inverse_transformer.transform(
        center.x,
        center.y
    )
)

# =====
# DATA
# =====

data = {
    "site":
        site_path,

    "obstacles":
        obstacle_paths,

    "slots":
        slot_paths,

    "aisles":
        aisle_paths,

    "spine":
        spine_path,

    "entry":
        entry_latlon,

    "center": {
        "lat":
            center_lat,

        "lng":
            center_lon
    },

    "n_slots":
        len(slot_paths),

    "angle":
        FINAL_LAYOUT[

```

```

        "angle"
    ],

    "access_side":
        FINAL_LAYOUT[
            "access_side"
        ]
    }

data_json = json.dumps(
    data
)

# =====
# GOOGLE MAP HTML
# =====

html = f"""

<!DOCTYPE html>

<html>

<head>

<style>

html, body {{
    margin: 0;
    padding: 0;
    width: 100%;
    height: 100%;
}}

#map {{
    width: 100%;
    height: 650px;
}}

#panel {{

    position: absolute;

    top: 15px;
    left: 15px;

    z-index: 999;

    background: white;

    padding: 15px;

    border-radius: 10px;

    font-family: Arial;

    box-shadow:
        0 2px 10px
        rgba(0,0,0,0.3);
}}

.legend span {{

    display: inline-block;

    width: 12px;
    height: 12px;

    margin-right: 6px;

    vertical-align: middle;
}}

```

```

</style>

</head>

<body>

<div id="panel">

<b>FINAL PARKING LAYOUT</b>

<br><br>

Reachable slots:
<b id="n"></b>

<br>

Angle:
<b id="ang"></b>°

<br>

Access:
<b id="side"></b>

<br><br>

<div class="legend">
<span style="background:#00ff00"></span>
Site boundary
</div>

<div class="legend">
<span style="background:#ea580c"></span>
Obstacles
</div>

<div class="legend">
<span style="background:#2563eb"></span>
Parking slots
</div>

<div class="legend">
<span style="background:#9333ea"></span>
Access lane
</div>

<div class="legend">
<span style="background:#fbbf24"></span>
Entry / Exit
</div>

</div>

<div id="map"></div>

<script>

const DATA =
{data_json};

function initMap() {{

    document.getElementById(
        "n"
    ).innerText =
        DATA.n_slots;

    document.getElementById(
        "ang"
    ).innerText =
        DATA.angle_deg;

```

```

DATA.angle.toFixed(1),

document.getElementById(
  "side"
).innerText =
  DATA.access_side;

const map =
  new google.maps.Map(
    document.getElementById(
      "map"
    ),
    {{
      center:
        DATA.center,

      zoom:
        20,

      mapTypeId:
        "satellite",

      tilt:
        0,

      streetViewControl:
        false,

      fullscreenControl:
        true

    }}
  );

// =====
// SITE
// =====

new google.maps.Polygon({{
  paths:
    DATA.site,

  strokeColor:
    "#00ff00",

  strokeWeight:
    3,

  fillOpacity:
    0,

  map:
    map
}});

// =====
// OBSTACLES
// =====

DATA.obstacles.forEach(
  function(o) {{
    new google.maps.Polygon({{
      paths:
        o,

      strokeColor:
        "#ea580c",

      strokeWeight:

```

```

        2,

        fillColor:
            "#ea580c",

        fillOpacity:
            0.5,

        map:
            map

    });

    }}
);

// =====
// ACCESS SPINE
// =====

new google.maps.Polygon({{

    paths:
        DATA.spine,

    strokeColor:
        "#9333ea",

    strokeWeight:
        1,

    fillColor:
        "#9333ea",

    fillOpacity:
        0.25,

    map:
        map

}});

// =====
// AISLES
// =====

DATA.aisles.forEach(
    function(a) {{

        new google.maps.Polygon({{

            paths:
                a,

            strokeColor:
                "#eab308",

            strokeWeight:
                1,

            fillOpacity:
                0,

            map:
                map

        }});

    }}
);

// =====
// SLOTS
// =====

```

```

DATA.slots.forEach(
  function(s) {{

    new google.maps.Polygon({{

      paths:
        s,

      strokeColor:
        "#2563eb",

      strokeWeight:
        1,

      fillColor:
        "#2563eb",

      fillOpacity:
        0.45,

      map:
        map

    }});

  }}
);

// =====
// ENTRY / EXIT
// =====

new google.maps.Marker({{

  position:
    DATA.entry,

  map:
    map,

  icon: {{

    path:
      google.maps.SymbolPath.CIRCLE,

    scale:
      8,

    fillColor:
      "#fbbf24",

    fillOpacity:
      1,

    strokeColor:
      "#000",

    strokeWeight:
      1

  }},

  title:
    "Entry / Exit"

});

}}

</script>

<script
async
src="https://maps.googleapis.com/maps/api/js?key={GOOGLE_MAPS_API_KEY}&loading=async&callback=initMap">
</script>

```

```

</body>

</html>

"""

display(
    HTML(html)
)

print(
    "=====
"
)

print(
    "FINAL VERIFIED LAYOUT"
)

print(
    "=====
"
)

print(
    f"Angle      : "
    f"{FINAL_LAYOUT['angle']:.1f}°"
)

print(
    f"Access side : "
    f"{FINAL_LAYOUT['access_side']}"
)

print(
    f"Reachable slots : "
    f"{len(FINAL_LAYOUT['slots'])}"
)

print(
    f"Obstacles      : "
    f"{len(OBSTACLE_POLYGONS_UTM)}"
)

# =====
# =====
# CELL 7 – OPTIONAL SEMI-AUTOMATIC BOUNDARY REFINEMENT
# =====
# =====

!pip -q install opencv-python-headless requests

import math
import requests
import cv2
import numpy as np

# =====
# WEB MERCATOR
# =====

def _project(
    lat,
    lon
):
    siny = min(
        max(
            math.sin(
                lat *
                math.pi /
                180
            ),

```

```

        -0.9999
    ),
    0.9999
)

x = (
    0.5
    +
    lon / 360
)

y = (
    0.5
    -
    math.log(
        (1 + siny)
        /
        (1 - siny)
    )
    /
    (4 * math.pi)
)

return (
    x,
    y
)

# =====
# LAT/LON → PIXEL
# =====

def latlon_to_pixel(
    lat,
    lon,
    center_lat,
    center_lon,
    zoom,
    img_w,
    img_h,
    scale=1
):
    world_w = (
        256 *
        (2 ** zoom)
    )

    cx, cy = _project(
        center_lat,
        center_lon
    )

    x, y = _project(
        lat,
        lon
    )

    px = (
        (x - cx)
        *
        world_w
        *
        scale
        +
        img_w / 2
    )

    py = (
        (y - cy)
        *
        world_w
        *
        scale
        +
        img_h / 2
    )

```

```

    return (
        px,
        py
    )

# =====
# PIXEL → LAT/LON
# =====

def pixel_to_latlon(
    px,
    py,
    center_lat,
    center_lon,
    zoom,
    img_w,
    img_h,
    scale=1
):
    world_w = (
        256 *
        (2 ** zoom)
    )

    cx, cy = _project(
        center_lat,
        center_lon
    )

    x = (
        (px - img_w / 2)
        /
        (world_w * scale)
        +
        cx
    )

    y = (
        (py - img_h / 2)
        /
        (world_w * scale)
        +
        cy
    )

    lon = (
        x - 0.5
    ) * 360

    n = (
        math.pi
        -
        2 *
        math.pi *
        y
    )

    lat = (
        180 /
        math.pi *
        math.atan(
            math.sinh(n)
        )
    )

    return (
        lat,
        lon
    )

# =====
# SETTINGS
# =====

```

```

IMG_SIZE = 640

SCALE = 2

ZOOM = 20

# =====
# CENTER
# =====

center_lat = float(
    np.mean([
        p["lat"]
        for p
        in boundary_points
    ])
)

center_lon = float(
    np.mean([
        p["lng"]
        for p
        in boundary_points
    ])
)

# =====
# GOOGLE STATIC MAP
# =====

static_url = (

    "https://maps.googleapis.com/maps/api/staticmap"

    f"?center="
    f"{center_lat},{center_lon}"

    f"&zoom={ZOOM}"

    f"&size="
    f"{IMG_SIZE}x{IMG_SIZE}"

    f"&scale={SCALE}"

    f"&maptype=satellite"

    f"&key="
    f"{GOOGLE_MAPS_API_KEY}"

)

resp = requests.get(
    static_url,
    timeout=20
)

resp.raise_for_status()

img_bytes = np.frombuffer(
    resp.content,
    dtype=np.uint8
)

img = cv2.imdecode(
    img_bytes,
    cv2.IMREAD_COLOR
)

img_h, img_w = (
    img.shape[:2]

```

```

)

print(
    f"Downloaded satellite crop: "
    f"{img_w}x{img_h}"
)

# =====
# CLICKED BOUNDARY → PIXELS
# =====

clicked_px = np.array([

    latlon_to_pixel(
        p["lat"],
        p["lng"],
        center_lat,
        center_lon,
        ZOOM,
        img_w,
        img_h,
        SCALE
    )

    for p
    in boundary_points

], dtype=np.int32)

# =====
# CHECK IMAGE BOUNDS
# =====

if (

    clicked_px[:, 0].min() < 0

    or

    clicked_px[:, 0].max() > img_w

    or

    clicked_px[:, 1].min() < 0

    or

    clicked_px[:, 1].max() > img_h

):

    print(
        "⚠ Boundary is outside "
        "satellite crop."
    )

# =====
# GRAB CUT MASK
# =====

mask = np.zeros(
    (
        img_h,
        img_w
    ),
    np.uint8
)

mask[:] = cv2.GC_BGD

# =====
# PROBABLE FOREGROUND REGION

```

```

# =====

pad = 25

x0 = max(
    clicked_px[:, 0].min()
    - pad,
    0
)

y0 = max(
    clicked_px[:, 1].min()
    - pad,
    0
)

x1 = min(
    clicked_px[:, 0].max()
    + pad,
    img_w - 1
)

y1 = min(
    clicked_px[:, 1].max()
    + pad,
    img_h - 1
)

mask[
    y0:y1,
    x0:x1
] = cv2.GC_PR_BGD

# =====
# EXACT CLICK POLYGON
# =====

cv2.fillPoly(
    mask,
    [clicked_px],
    cv2.GC_FGD
)

# =====
# SHRUNK FOREGROUND
# =====

shrunk = (
    clicked_px
    -
    clicked_px.mean(
        axis=0
    )
) * 0.7 + (
    clicked_px.mean(
        axis=0
    )
)

cv2.fillPoly(
    mask,
    [
        shrunk.astype(
            np.int32
        )
    ],
    cv2.GC_FGD
)

# =====
# GRAB CUT

```

```

# =====

bgd_model = np.zeros(
    (1, 65),
    np.float64
)

fgd_model = np.zeros(
    (1, 65),
    np.float64
)

cv2.grabCut(
    img,
    mask,
    None,
    bgd_model,
    fgd_model,
    5,
    cv2.GC_INIT_WITH_MASK
)

# =====
# FOREGROUND MASK
# =====

fg_mask = np.where(

    (
        mask ==
        cv2.GC_FGD
    )

    |

    (
        mask ==
        cv2.GC_PR_FGD
    ),

    255,
    0

).astype(
    np.uint8
)

fg_mask = cv2.morphologyEx(

    fg_mask,

    cv2.MORPH_CLOSE,

    np.ones(
        (5, 5),
        np.uint8
    )

)

# =====
# EXTRACT CONTOUR
# =====

contours, _ = cv2.findContours(

    fg_mask,

    cv2.RETR_EXTERNAL,

    cv2.CHAIN_APPROX_SIMPLE

)

```

```

if not contours:

    print(
        "GrabCut found no region."
    )

    refined_points = None

else:

    largest = max(
        contours,
        key=cv2.contourArea
    )

    epsilon = (
        0.01 *
        cv2.arcLength(
            largest,
            True
        )
    )

    approx = cv2.approxPolyDP(

        largest,

        epsilon,

        True

    ).reshape(
        -1,
        2
    )

    refined_points = [

        {
            "lat": lat,
            "lng": lon
        }

        for (
            px,
            py
        )
        in approx

        for lat, lon
        in [
            pixel_to_latlon(

                px,
                py,

                center_lat,
                center_lon,

                ZOOM,

                img_w,
                img_h,

                SCALE

            )
        ]
    ]

```

```

orig_area_px = (
    cv2.contourArea(
        clicked_px
    )
)

refined_area_px = (
    cv2.contourArea(
        approx.astype(
            np.int32
        )
    )
)

change_pct = (
    100 *
    (
        refined_area_px
        -
        orig_area_px
    )
    /
    max(
        orig_area_px,
        1
    )
)

print(
    "\n=====
"
)

print(
    "GRAB CUT BOUNDARY SUGGESTION"
)

print(
    "=====
"
)

print(
    f"Original points : "
    f"{len(boundary_points)}"
)

print(
    f"Refined points : "
    f"{len(refined_points)}"
)

print(
    f"Area change      : "
    f"{change_pct:+.1f}%"
)

print(
    "\n⚠ This is only a heuristic suggestion."
)

# =====
# VISUAL CHECK
# =====

fig, ax = plt.subplots(
    1,
    2,
    figsize=(14, 7)
)

# Original
cv2.imshow('

```

```

plt.imshow(
    cv2.cvtColor(
        img,
        cv2.COLOR_BGR2RGB
    )
)

ax[0].plot(
    *zip(
        *np.vstack([
            clicked_px,
            clicked_px[0]
        ])
    ),
    color="lime",
    linewidth=2
)

ax[0].set_title(
    "Original Click Boundary"
)

ax[0].axis(
    "off"
)

# GrabCut
ax[1].imshow(
    fg_mask,
    cmap="gray"
)

if contours:
    approx_closed = np.vstack([
        approx,
        approx[0]
    ])

    ax[1].plot(
        approx_closed[:, 0],
        approx_closed[:, 1],
        "r",
        linewidth=2
    )

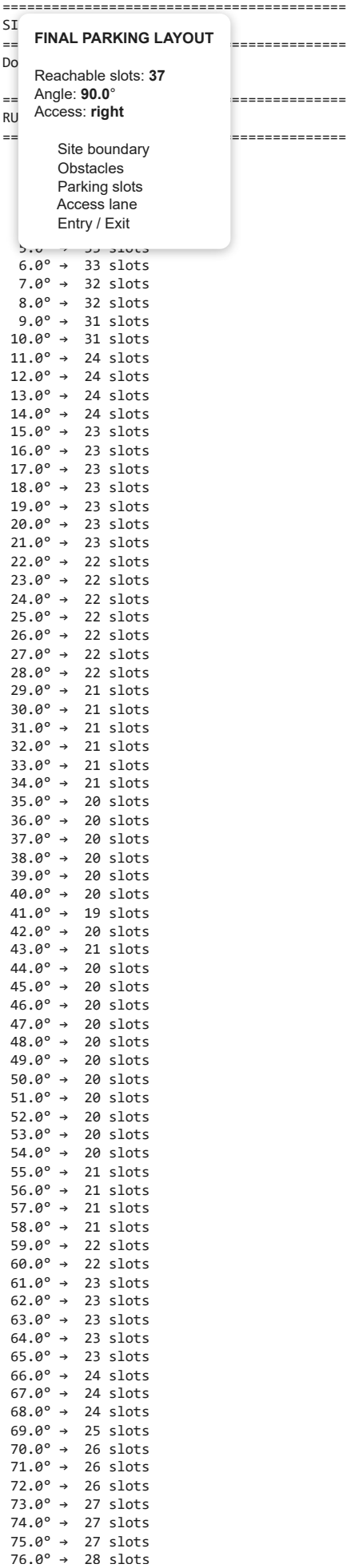
ax[1].set_title(
    "GrabCut Boundary Suggestion"
)

ax[1].axis(
    "off"
)

plt.show()

# =====
# END
# =====
print("\n=====")
print("PARKING OPTIMIZER V2 COMPLETE")
print("=====")

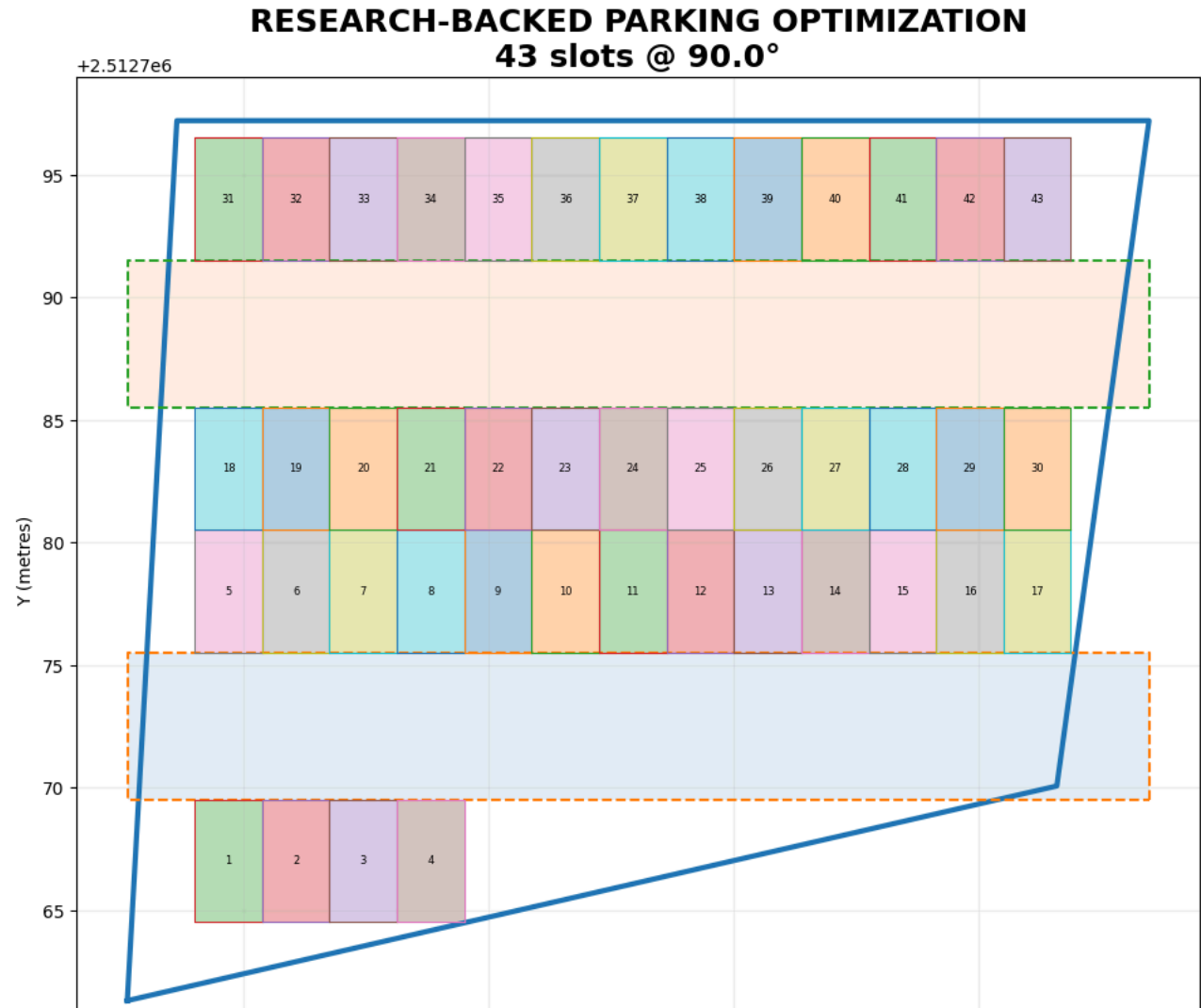
```

```
77.0° → 29 slots
78.0° → 29 slots
79.0° → 30 slots
80.0° → 31 slots
81.0° → 32 slots
82.0° → 32 slots
83.0° → 33 slots
84.0° → 35 slots
85.0° → 36 slots
86.0° → 37 slots
87.0° → 38 slots
88.0° → 40 slots
89.0° → 41 slots
90.0° → 43 slots

=====
OPTIMAL RESULT
=====
Best angle      : 90.0°
Best slot count : 43
Aisle width     : 6.00 m
Module depth    : 16.00 m
Plot area       : 1209.94 m²
23 m²/ECS check : 52.6

=====
TOP CONFIGURATIONS
=====
90.0° → 43 slots
89.0° → 41 slots
88.0° → 40 slots
0.0° → 39 slots
87.0° → 38 slots
1.0° → 37 slots
86.0° → 37 slots
2.0° → 36 slots
85.0° → 36 slots
3.0° → 35 slots
```



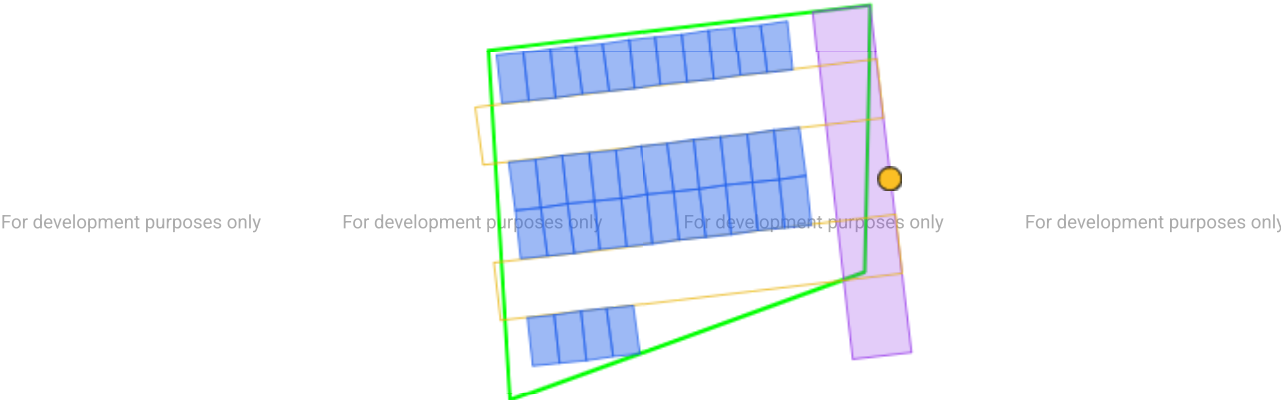


```
=====
ENTRY / EXIT + REACHABILITY
=====
Access from left edge → 35 reachable slots (2/2 aisle bays connected)
Access from right edge → 37 reachable slots (2/2 aisle bays connected)
```

```
-----
Best access side      : right
Before reachability   : 43
After reachability    : 37
Δ 6 slots dropped.

✓ FINAL_LAYOUT ready: 37 reachable slots.
```

For development purposes only For development purposes only For development purposes only For development purposes only



Google For development purposes only For development purposes only For development purposes only Imagery ©2026 Airbus, Maxar Technologies Report a map error

```
=====
FINAL VERIFIED LAYOUT
=====
Angle      : 90.0°
Access side : right
Reachable slots : 37
Obstacles   : 0
```

```
-----
HTTPError                                Traceback (most recent call last)
/tmp/ipykernel_2045/2415675453.py in <cell line: 0>()
    1994 )
    1995
-> 1996 resp.raise_for_status()
    1997
    1998
```

```
/usr/local/lib/python3.12/dist-packages/requests/models.py in raise_for_status(self)
```

```
    1024
    1025         if http_error_msg:
-> 1026             raise HTTPError(http_error_msg, response=self)
    1027
    1028     def close(self):
```

HTTPError: 403 Client Error: Forbidden for url: <https://maps.googleapis.com/maps/api/staticmap?center=22.720019975329794,75.858232865497&zoom=20&size=640x640&scale=2&maptype=satellite&key=AIzaSyCOFo65FvdvFDC0chQq9AWDdyJehsst>

```

# =====
# CELL 5 - IMPROVED ACCESS + SLOT REACHABILITY
# =====

from shapely.geometry import Point, LineString
from shapely.ops import unary_union
from shapely.affinity import translate

# =====
# PARAMETERS
# =====

ENTRY_WIDTH = 6.0

CONNECTOR_WIDTH = 6.0

REACH_TOLERANCE = 0.20

# =====
# CREATE ENTRY CONNECTOR
# =====

def create_entry_connector(
    polygon,
    aisles,
    side,
    width=6.0
):
    minx, miny, maxx, maxy = (
        polygon.bounds
    )

    site_height = maxy - miny

    # -----
    # ENTRY POINT
    # -----

    if side == "right":

        entry_x = maxx

    else:

        entry_x = minx

    # -----
    # Find aisle closest to entry
    # -----

    if not aisles:

        return None, None, None

    best_aisle = min(
        aisles,
        key=lambda a:
            a.distance(
                Point(
                    entry_x,
                    (miny + maxy) / 2
                )
            )
    )

    # -----
    # Find closest point on aisle
    # -----

```

```

    entry_point = Point(
        entry_x,
        best_aisle.centroid.y
    )

    aisle_point = best_aisle.centroid

    # -----
    # Build access corridor
    # -----

    connector_line = LineString([
        entry_point.coords[0],
        aisle_point.coords[0]
    ])

    connector = (
        connector_line
        .buffer(
            width / 2,
            cap_style=2
        )
    )

    # Keep connector inside site
    connector = connector.intersection(
        polygon
    )

    return (
        connector,
        entry_point,
        best_aisle
    )

# =====
# CHECK SLOT → AISLE ACCESS
# =====

def slot_reaches_aisle(
    slot,
    aisles,
    tolerance=0.20
):

    if not aisles:

        return False

    # Slot centroid
    center = slot.centroid

    # Distance to nearest aisle
    nearest = min(
        aisles,
        key=lambda a:
            center.distance(a)
    )

    distance = center.distance(
        nearest
    )

    # -----
    # Important:
    # A parking stall is considered accessible
    # when its edge is sufficiently close to an aisle.

```

```

# -----

edge_distance = slot.distance(
    nearest
)

return (
    edge_distance
    <=
    tolerance
)

# =====
# TEST ACCESS SIDE
# =====

def evaluate_access_side(
    polygon,
    slots,
    aisles,
    side
):
    (
        connector,
        entry_point,
        entry_aisle
    ) = create_entry_connector(
        polygon,
        aisles,
        side,
        CONNECTOR_WIDTH
    )

    if connector is None:

        return {
            "side": side,
            "connector": None,
            "entry_point": None,
            "entry_aisle": None,
            "slots": [],
            "score": 0
        }

# -----
# Determine slots accessible from aisle network
# -----

reachable = []

for slot in slots:

    if slot_reaches_aisle(
        slot,
        aisles,
        REACH_TOLERANCE
    ):

        reachable.append(
            slot
        )

# -----
# Entry connected to selected aisle?
# -----

entry_connected = (
    connector.intersects(
        entry_aisle
    )

```